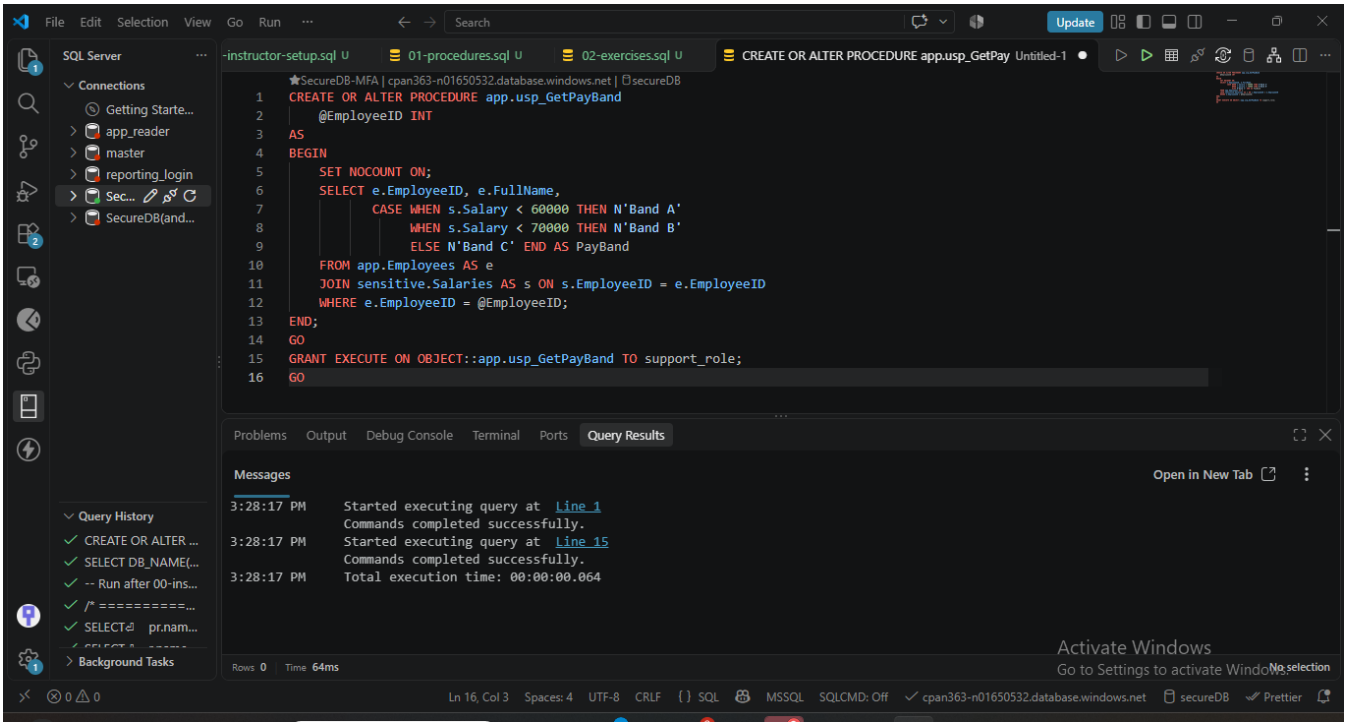
CPAN 363 — DATABASE SECURITY · LABORATORY 5

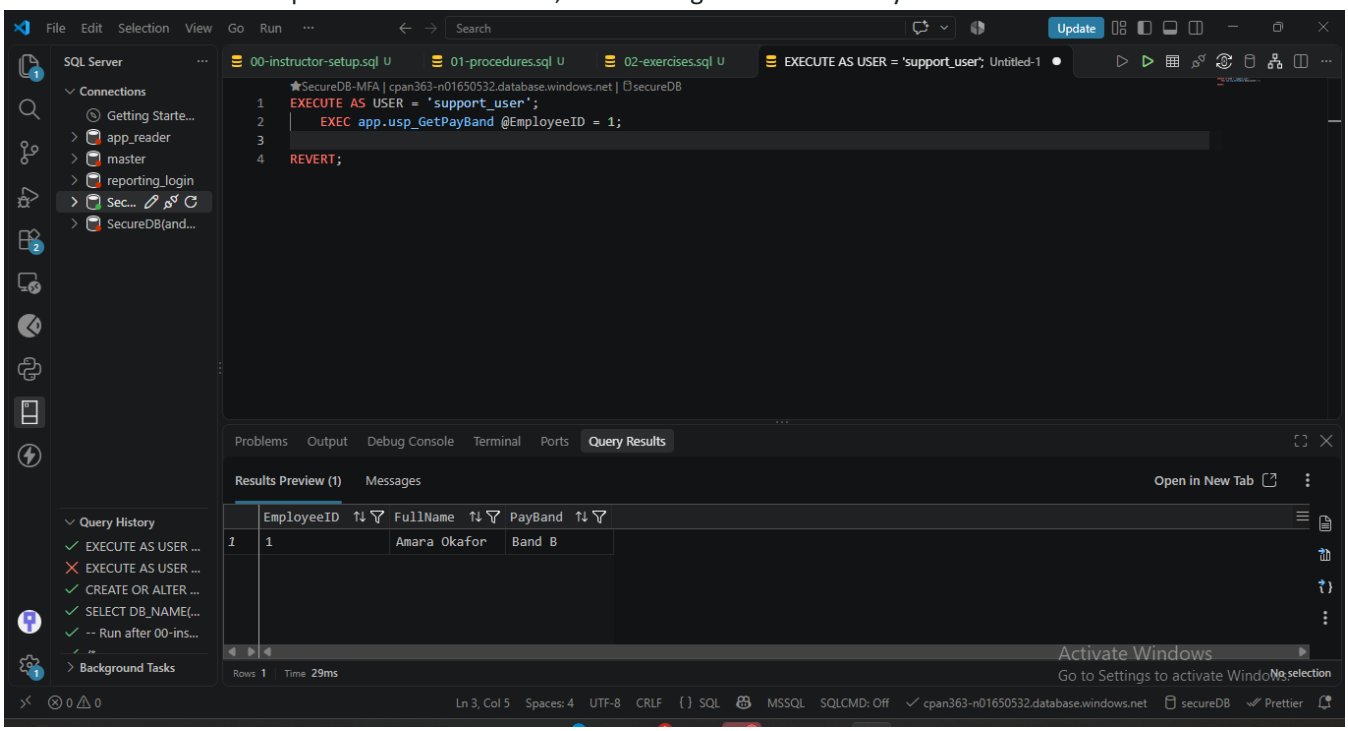
Stored Procedures and SQL Injection

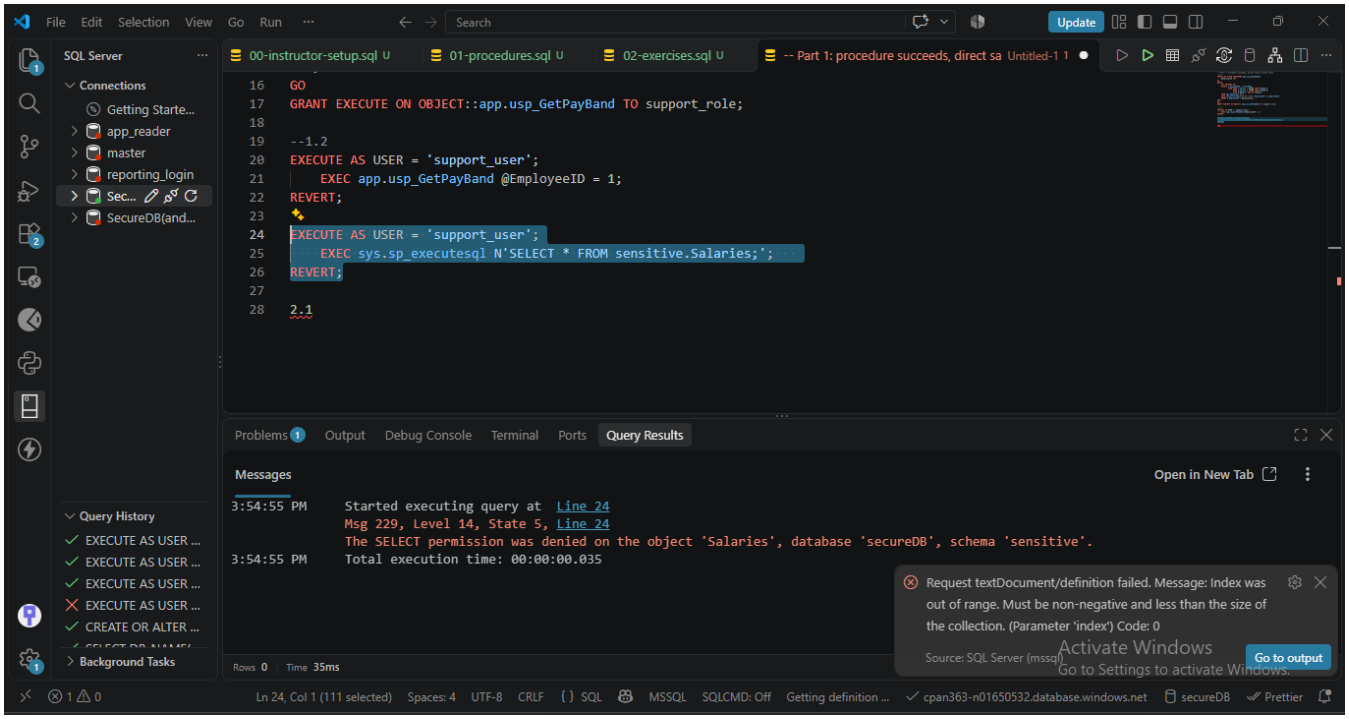
Part 1 — Build a safe door

1.1 Create a procedure that reads the sensitive table but returns only safe information.



1.2 Test both halves: the procedure should work, and reading the table directly should not.





1.3 Write two sentences explaining why the support user can get a pay band but cannot get a salary.

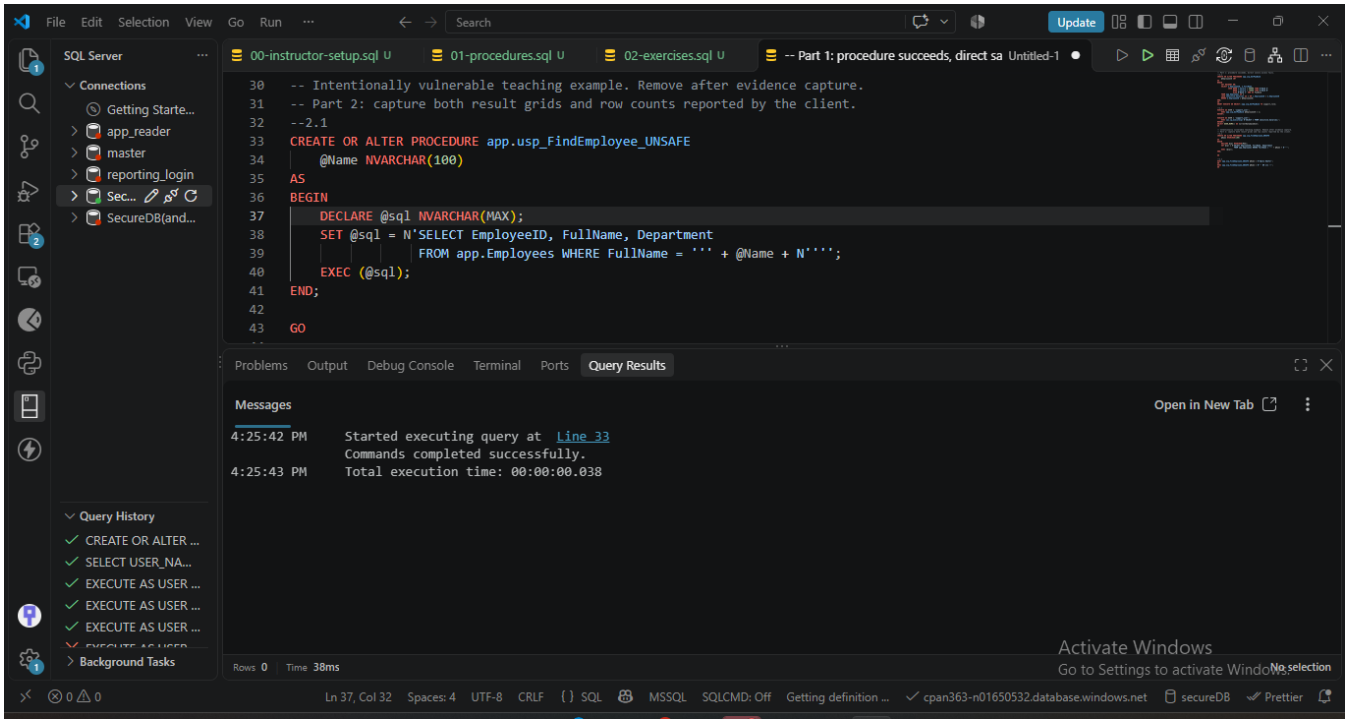
The support user can get a pay band because its role has EXECUTE permission on the stored procedure, and ownership chaining allows the procedure to read the salary table while returning only the pay band. The user cannot read actual salaries directly because its role is explicitly denied SELECT permission on the sensitive schema.

> **Checkpoint**
> A user with no access to the sensitive table can still get the information their job needs.

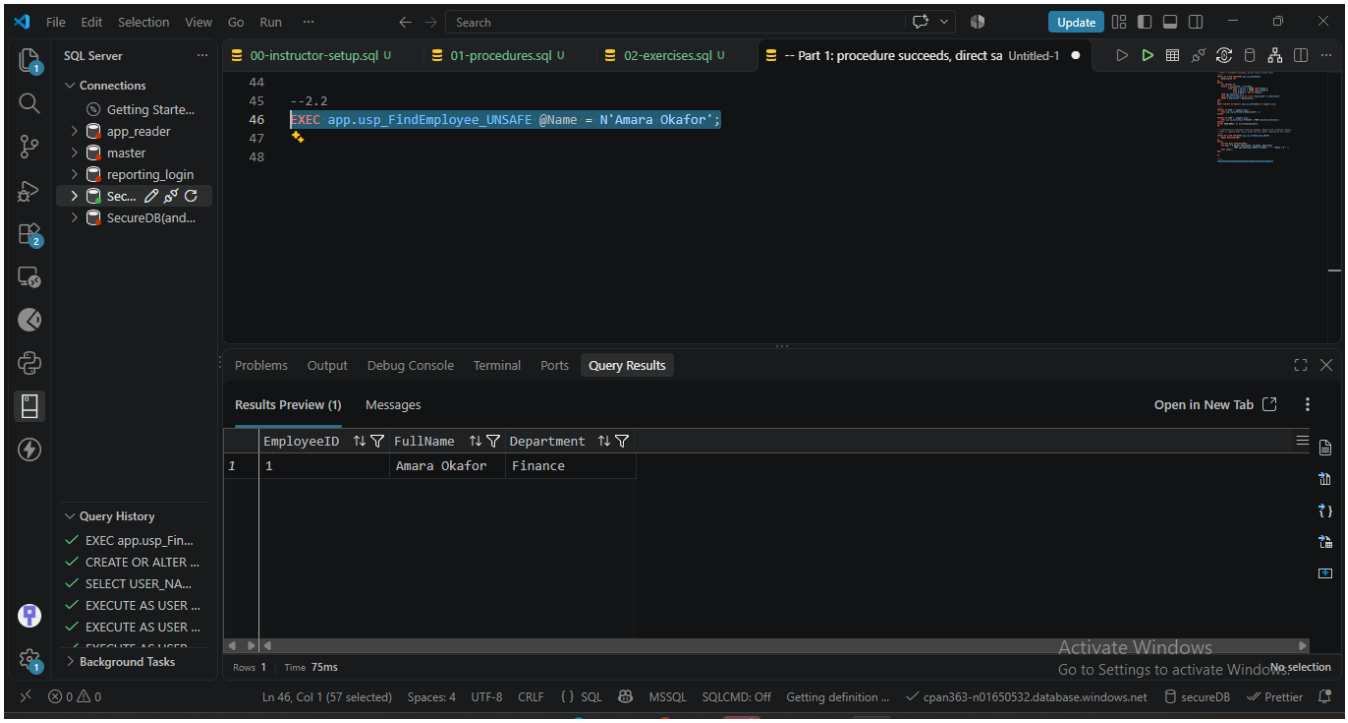
Part 2 — Write an unsafe procedure

Everything in this part is done only on your own database.

2.1 Create a search procedure that joins the input onto the SQL command.

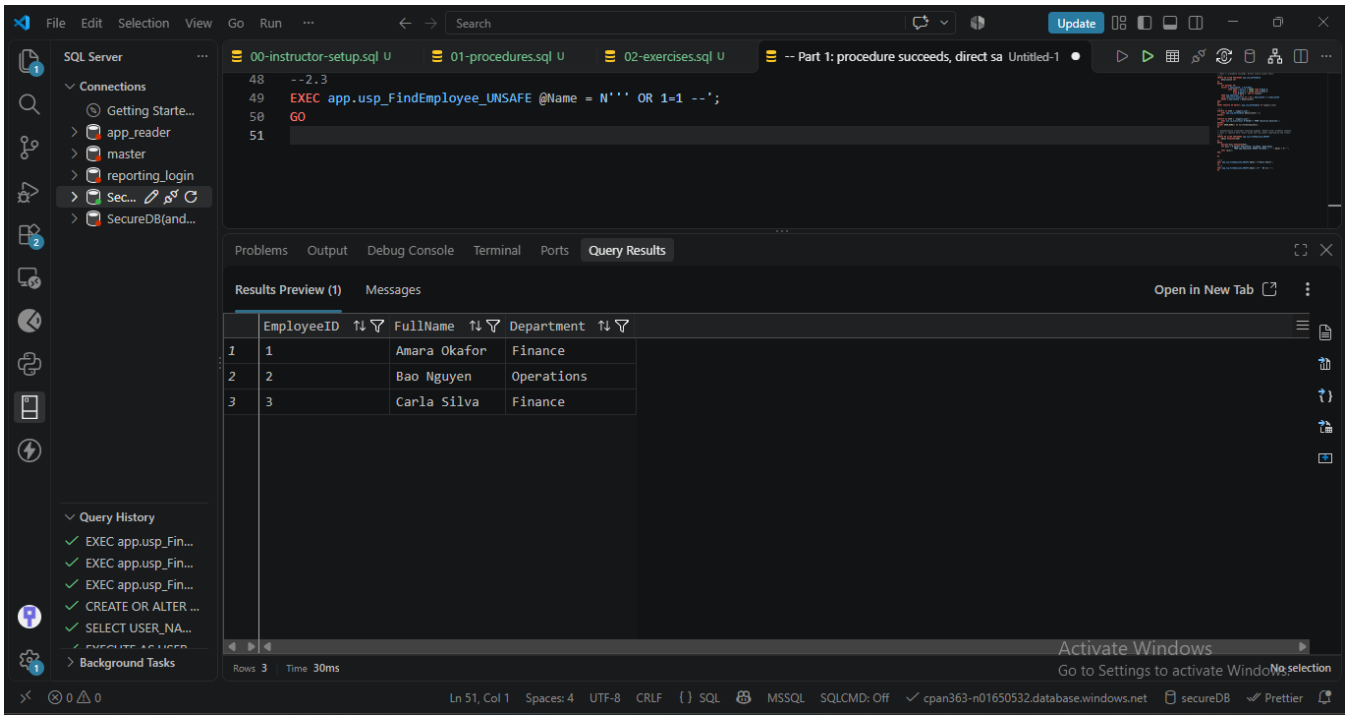


2.2 Run it normally first, so you have something to compare against.



Capture: *The result and how many rows came back.*

2.3 Now run it with input designed to change the query.



> **Note**
> The input ended the text early, added a condition that is always true, and commented out the rest. The query now returns every row.

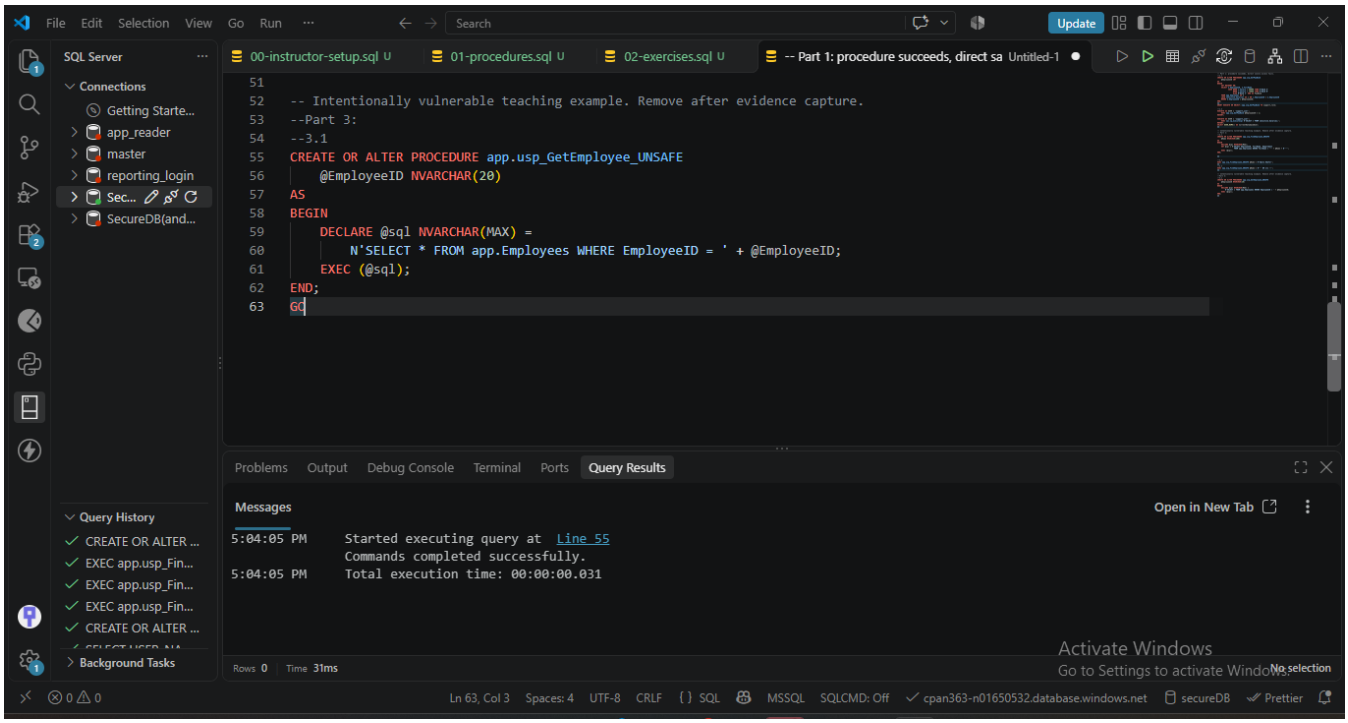
Capture: *The result and how many rows came back. Compare this number with the one above.*

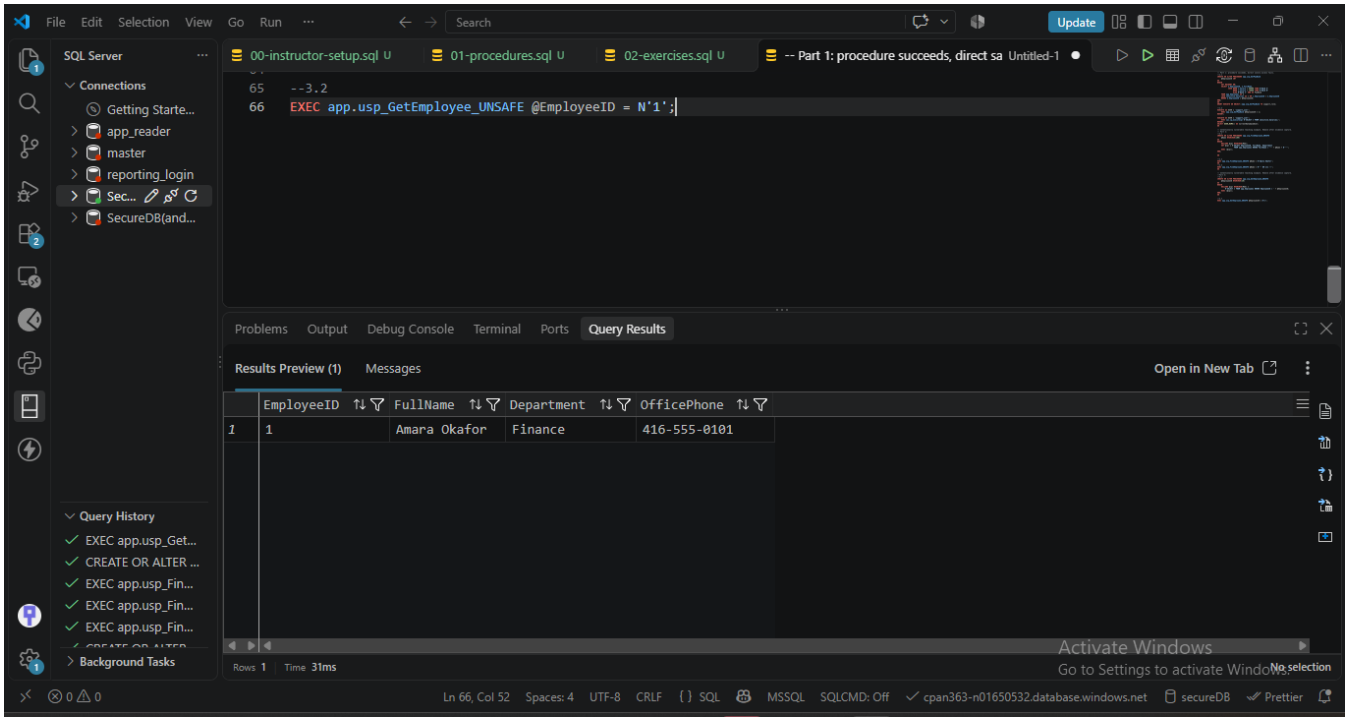
2.4 In your own words, explain in three or four sentences what happened.

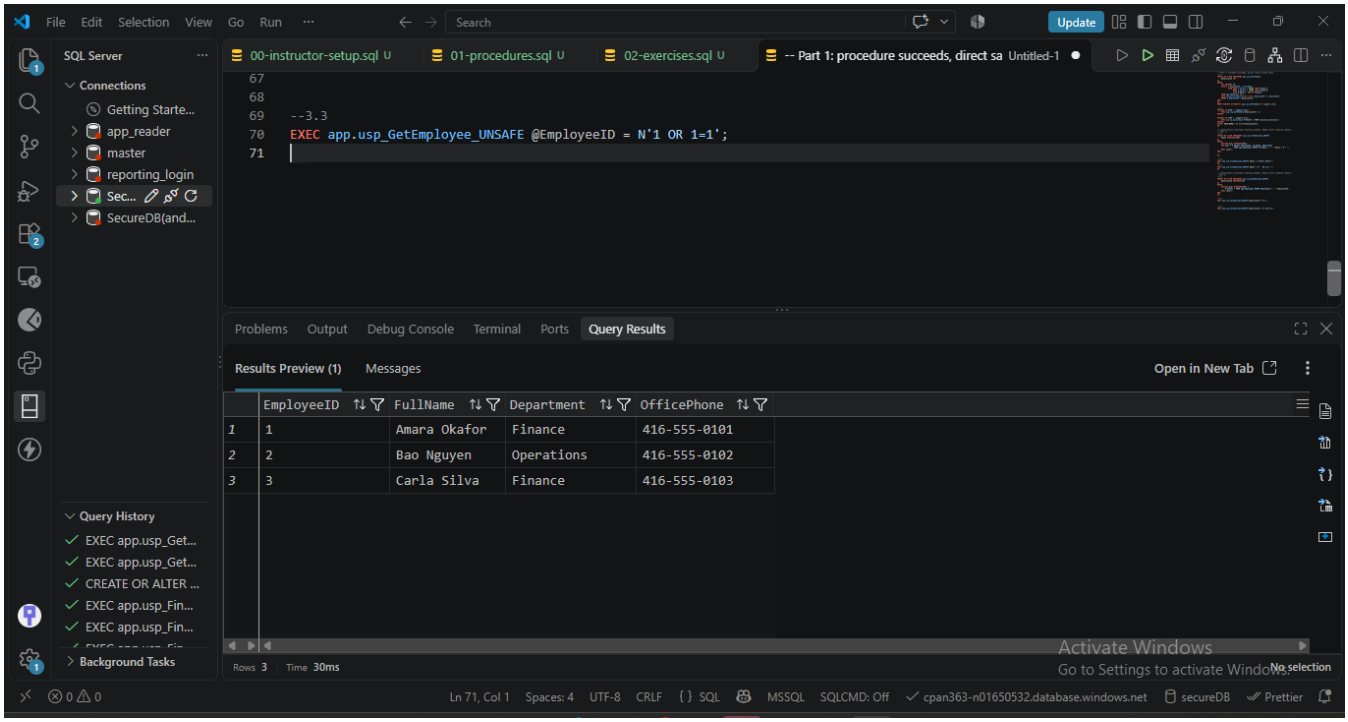
The unsafe procedure joins the supplied name directly into the SQL command. The injected quote closes the name value, OR 1=1 makes the condition true for every row, and - comments out the remaining quote. This changes the query's meaning instead of searching for a literal name. As a result, the injected search returned all three employees instead of only Amara Okafor.

Part 3 — Try it without a quote mark

3.1 Create a second unsafe procedure that takes a number as text.





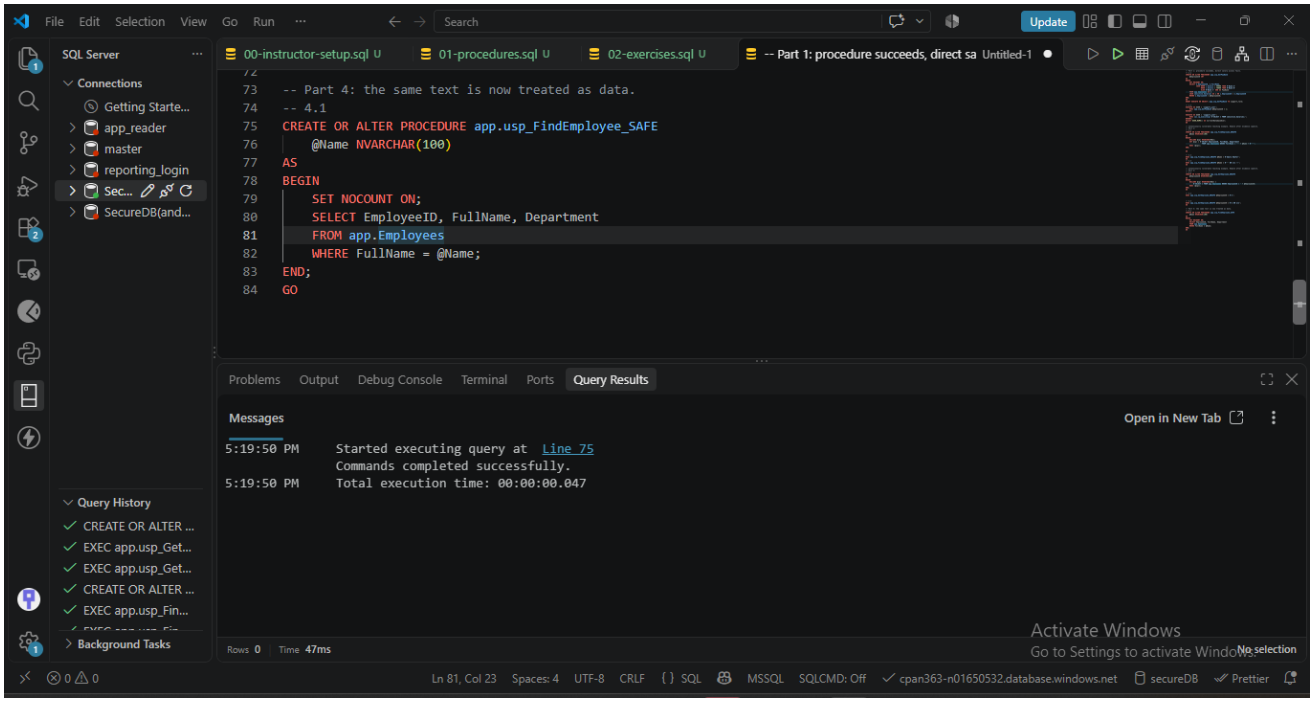


> **Note**
> There was no quote mark to escape here. This is why 'just handle the quotes' is not a real fix.

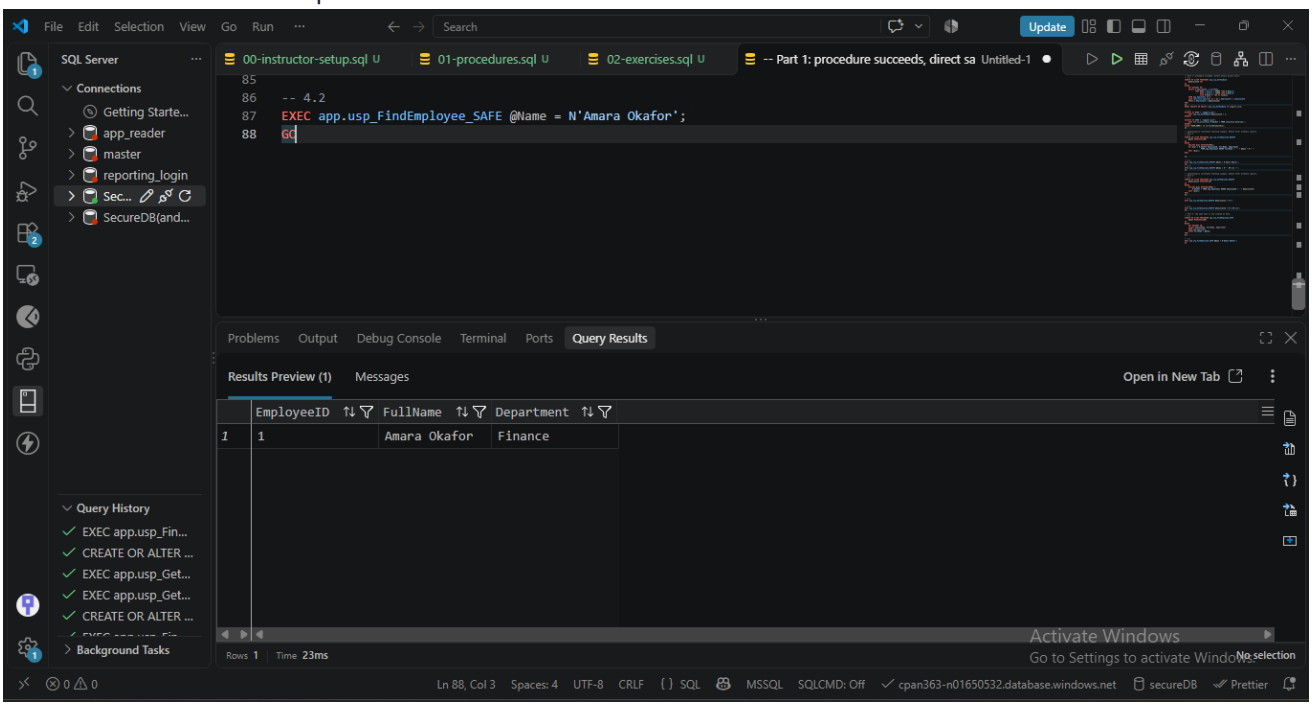
Capture: *Both results.*

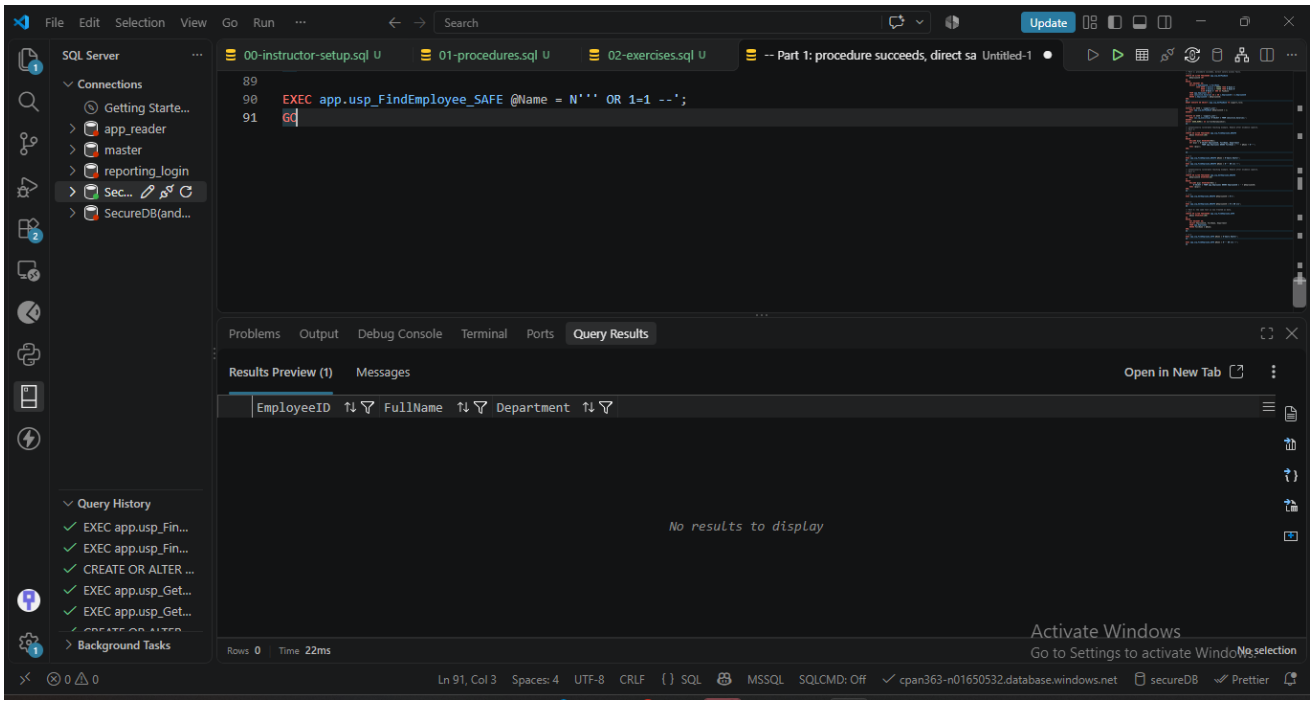
Part 4 — Fix both procedures

4.1 Rewrite the search procedure using a parameter and no dynamic SQL.



4.2 Send the same attack input to the safe version.

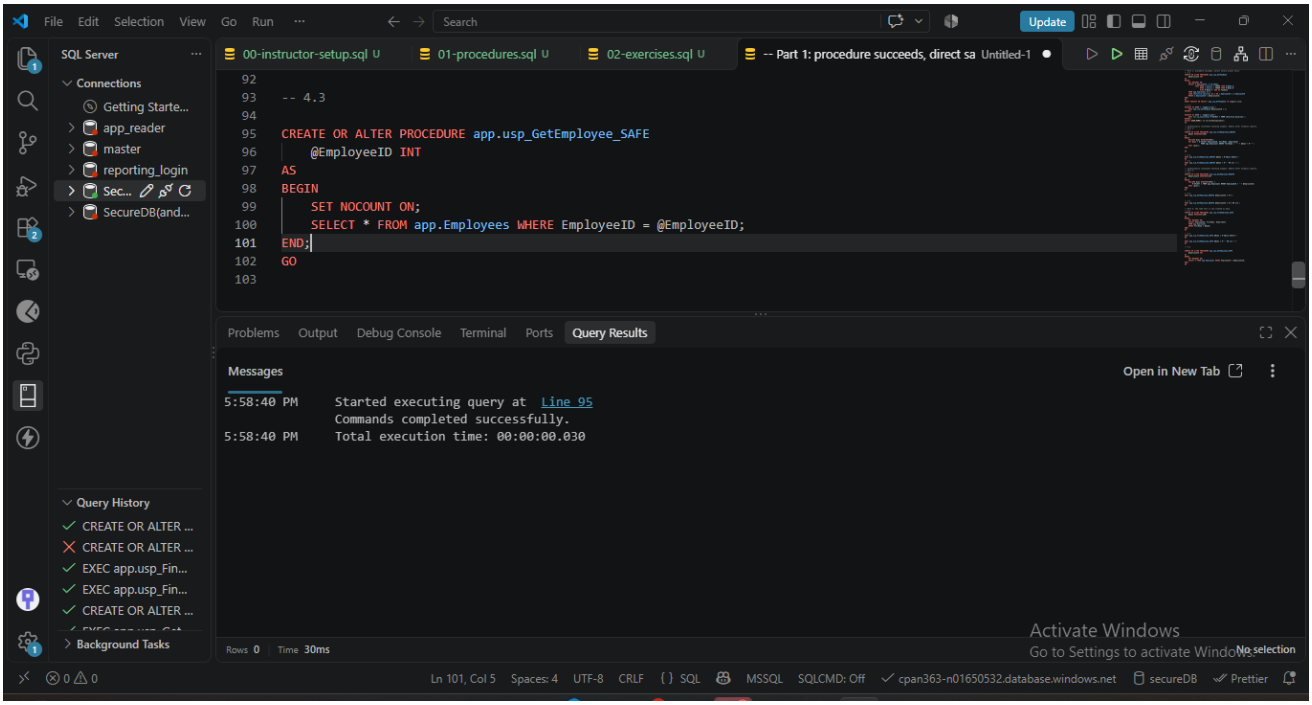


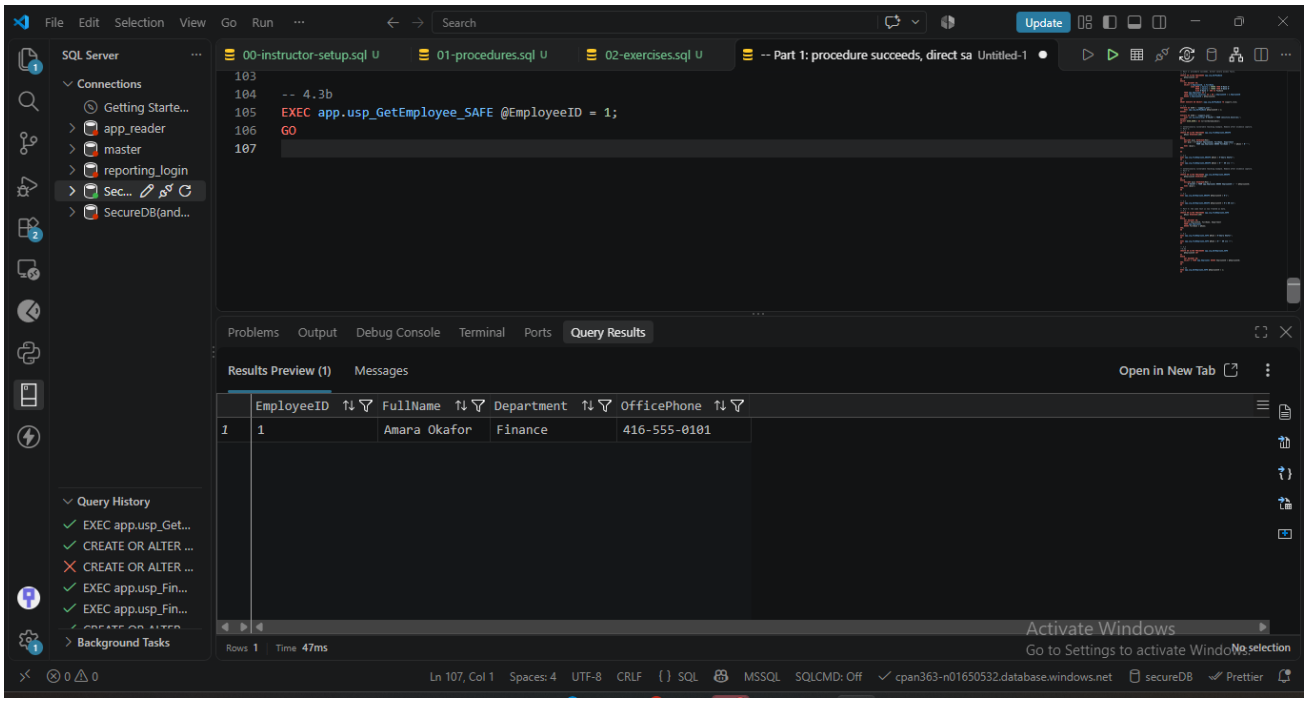


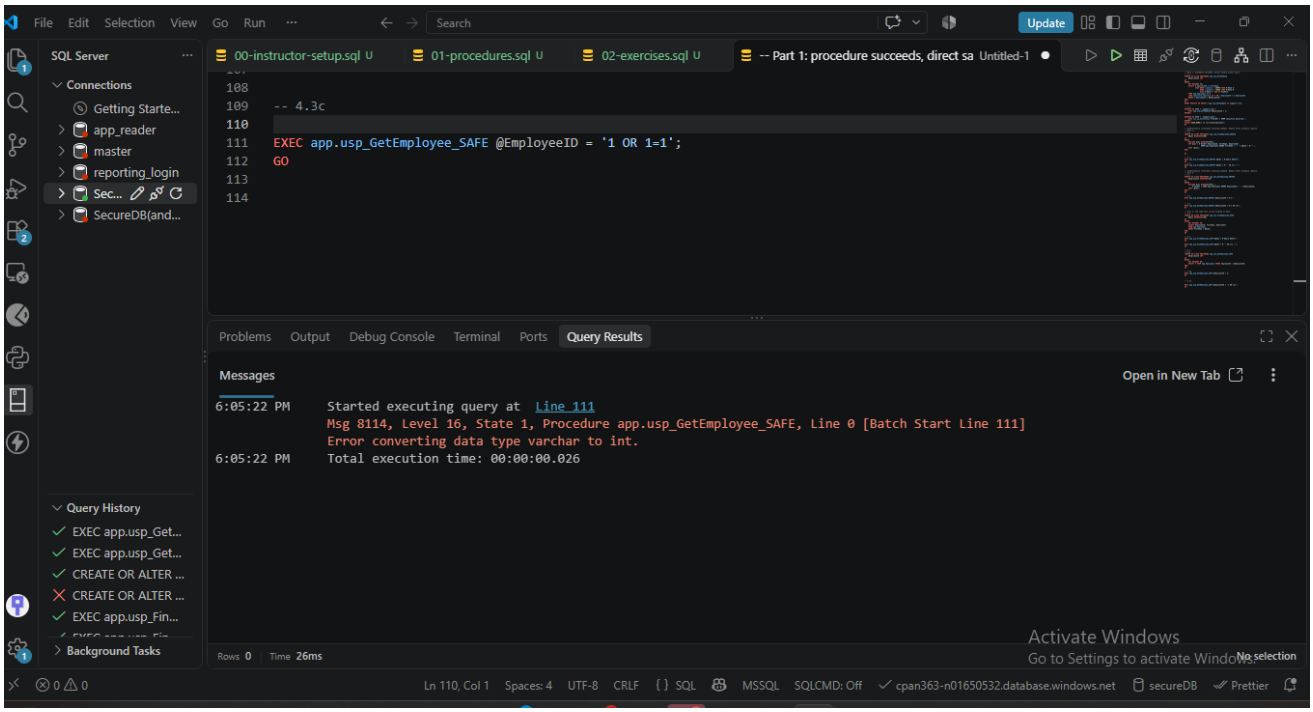
> **Note**
> The database searched for an employee whose name is literally ' OR 1=1 -- and did not find one. The input stayed a value.

Capture: *Both results. The attack input should now return zero rows.*

4.3 Fix the number procedure by using the right data type.







Capture: *Both results, including the conversion error.*

4.4 Write two or three sentences explaining why a parameter cannot be injected.

The fixed procedures use parameters directly in static SQL, so the supplied values cannot change the query's structure. The name procedure treats the injection text as a literal name and returns no matches, while the integer parameter rejects 1 OR 1=1 with a conversion error. A procedure parameter alone is not enough to prevent injection if its value is later concatenated into executable SQL.

> **Checkpoint**
> You have evidence that the same input returns every row from the unsafe version and nothing from the safe version.

Part 5 — Review and tidy up

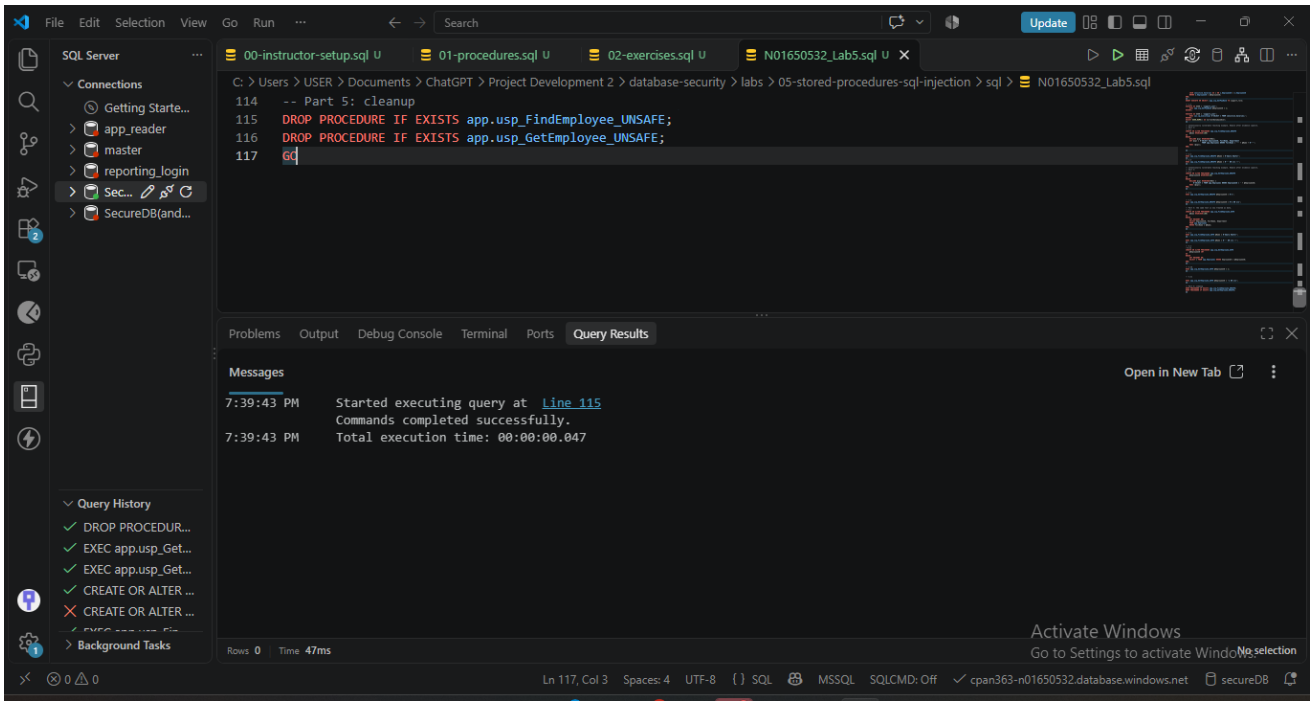
5.1 Fill in this checklist for both versions of the search procedure.

Check	UNSAFE version	SAFE version
Joins input with a plus sign?	Yes	No
Uses EXEC(@sql)?	Yes	No
Uses a parameter?	Yes, but concatenates its value into SQL	Yes, used directly as a value
Correct data type used?	Yes-NVARCHAR(100) for a name	Yes-NVARCHAR(100) for a name
Could this be injected?	Yes	No-the query is static and parameterized

5.2 Answer in a few sentences: how did the role model you built in Labs 3 and 4 limit what an attacker could have done here?

The role model limited users to the permissions needed for their jobs, reducing the damage an attacker could cause through SQL injection. For example, the support role could read employee information but was denied direct access to the sensitive salary table. Injection would still be restricted by the permissions of the execution context; because these injection tests ran as the administrator, they had broader access than a support user would.

5.3 Delete the unsafe procedures so they do not survive into later labs.



5.4 Sign and include the statement below with your submission.

I confirm that all testing in this lab was performed only against the Azure SQL database in my own subscription, created for this course.

Name: Student ID:

Signature: CAO Date: